Supplementary Information
On the Conservation Laws and Pathway Realizability of Biochemical Reaction Networks

Thanh-An Pham[1] and Tieu-Long Phan[1,2*]

[1]Bioinformatics Group, Department of Computer Science & Interdisciplinary Center for Bioinformatics & School for Embedded and Composite Artificial Intelligence (SECAI), Leipzig University, Härtelstraße 16–18, D-04107, Leipzig, Germany.
[2]Department of Mathematics and Computer Science, University of Southern Denmark, DK-5230, Odense M, Denmark.

*Corresponding author(s). E-mail(s): tieu@bioinf.uni-leipzig.de;
Contributing authors: thanh-an.pham@uni-leipzig.de;

S1 Python interface example

Code Example S1 illustrates network construction, exact structural analysis, and discrete pathway verification through `SynKit`'s Python interface. The script builds a 14-species glycolysis model (with currency metabolites omitted), evaluates chemical reaction network theory (CRNT) invariants and deficiency, isolates conserved pools and minimal siphons, and verifies that one glucose token can reach two pyruvates via an exact 15-step firing sequence.

```python
from synkit.CRN import (build_native, native_backend, crnt_summary,
    conserved_moieties, find_siphons, siphon_persistence_details,
    PathwayRealizability, crn_to_sbml)
from Experiment.CRN.studies import build_kegg_crn, GLYCOLYSIS_FLUX

library = build_native("/tmp/synkit-crn", with_semiflows=True)
with native_backend(library):
    crn = build_kegg_crn("M00001", drop_currency=True)
    crn.n_species, crn.n_reactions                    # (14, 15)
    report = crnt_summary(crn)
    report.deficiency, report.is_weakly_reversible    # (1, False)
    len(conserved_moieties(crn)), len(find_siphons(crn)) # (3, 3)
    siphon_persistence_details(crn).persistence_ok    # False

    check = PathwayRealizability().load_syncrn_and_flow(
        crn, flow=GLYCOLYSIS_FLUX,
        initial_marking={"alpha-D-Glucose": 1},
        species="label", reaction="label")
    check.build_petri_net_from_flow()
    outcome = check.realizability_result()
    outcome.status.value, len(outcome.certificate) # ("realizable", 15)
    sbml_document = crn_to_sbml(crn)
```

Code Example S1: Python analysis of reduced `KEGG` glycolysis (`M00001`): network construction, exact structural analysis, discrete pathway verification, and `SBML` export.

S2 Reference-network validation and numerical comparison

Table S1 details the literature sources, targeted structural validation properties, and conservation-law audit for the twelve reference networks.

Table S1: Reference reaction networks: literature sources, targeted structural validation properties, and conservation-law numerical audit. SVD residual reports max $|\mathbf{S}^T B|$ for a floating-point basis B, Rounded invalid counts columns of round$(1000B)$ with non-zero integer residuals, and Exact residual evaluates `SynKit`'s integer basis. The open reactor has a zero-dimensional conservation space.

Network model	Source	Targeted property	SVD res.	Rounded inv.	Exact res.
Reversible isomerization	Feinberg [1], Ch. 3	Minimal weakly reversible, Deficiency Zero Theorem (positive branch)	1.1e-16	0	0
Irreversible isomerization	Feinberg [1], Ch. 3	Deficiency Zero Theorem (negative branch)	0.0e+00	0	0
Isomerization cycle	Feinberg [1], Ch. 3	Weak reversibility without individual reversible steps	2.2e-16	0	0
Three disjoint reversible pairs	Constructed test network	Disconnected linkage classes, with minimal semiflow supports isolating disjoint pairs rather than their union	1.1e-16	0	0
Disjoint pairs, non-unit stoichiometry	Constructed test network	As above with $E \rightleftharpoons 2F$: exact pool $2E + F$, which rounding can corrupt	1.1e-16	0	0
Michaelis-Menten	Michaelis & Menten, Feinberg [1]	Enzyme mechanism with two conserved pools, $\delta = 0$, non-weakly reversible	2.2e-16	1	0
Reversible Michaelis-Menten	Feinberg [1]	$\delta = 0$, with weak reversibility restored	1.7e-16	1	0
Edelstein bistable switch	Edelstein [2]	Minimal reversible $\delta = 1$ network with multiple steady states	0.0e+00	0	0
Futile cycle (single-site)	Wang & Sontag [3]	Distributive cycle with three independent pools	3.3e-16	3	0
Futile cycle (two-site)	Wang & Sontag [3]	Exact conservation in a multistationarity motif	5.3e-16	0	0
Shinar-Feinberg ACR motif	Shinar & Feinberg [4]	ACR motif, where a minimal siphon contains no P-semiflow support	1.1e-16	0	0
Open reactor $(0 \to A \to B \to 0)$	Constructed test network	Zero-complex handling, where cycle closure changes weak reversibility	0.0e+00	0	0

S2.1 Algebraic consistency and deficiency validation

We cross-checked deficiency using two formulations. With species-by-complex matrix $\mathbf{Y}$ and directed complex-graph incidence $\mathbf{I}_a$, we have $\mathbf{S} = \mathbf{Y}\mathbf{I}_a$ and

$$\delta \;=\; n - \ell - \operatorname{rank}(\mathbf{S}) \;=\; \operatorname{rank}(\mathbf{I}_a) - \operatorname{rank}(\mathbf{Y}\mathbf{I}_a). \tag{1}$$

Both formulations agreed for every benchmark, confirming consistency among complex partitioning, rank computation, and stoichiometric assembly. Fig. S1 illustrates the cross-check and the theorem verdicts on the Edelstein network.

S2.2 Numerical nullspaces and exact integer conservation laws

A conservation law is a non-zero integer vector y with $y^T\mathbf{S} = 0$. Floating-point nullspace routines return approximate bases, and rounding scaled vectors to integers is a common shortcut. We compared this heuristic

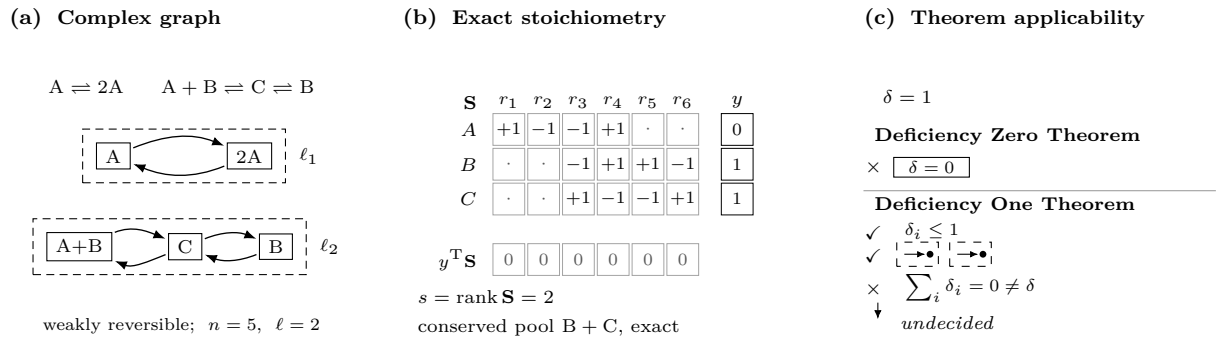


Fig. S1: Conservation structure and deficiency-theorem checks for the Edelstein network [2]. **(a)** Complex graph and its two linkage classes. **(b)** Stoichiometric matrix beside the primitive conservation vector y, whose product $y^{\mathsf{T}}\mathbf{S}$ vanishes over $\mathbb{Q}$ and certifies the conserved pool $[\mathrm{B}] + [\mathrm{C}]$. **(c)** Each classical deficiency theorem fails one hypothesis, leaving multistationarity undecided by deficiency alone.

with `SynKit`'s exact rational solver on the twelve reference networks (Table S1). SVD residuals were of order 10^{-16} on closed networks, but after scaling by 10^3 and rounding, the recovered vectors violated $y^T\mathbf{S} = 0$ on three models: Michaelis–Menten, reversible Michaelis–Menten, and the one-site futile cycle. `SynKit`'s exact elimination returned valid integer vectors in every case. **[To do S1.]** Replace the scale-and-round baseline with cdd or clingo enumeration on every network (main-text note 5; report B3).

S3 Computational cost across network families

Fig. S2 illustrates the three benchmark families. Fig. S3 reports the runtimes for the four structural analyses across these families, with numbers beside the random-sparse siphon points giving the number of minimal siphons.

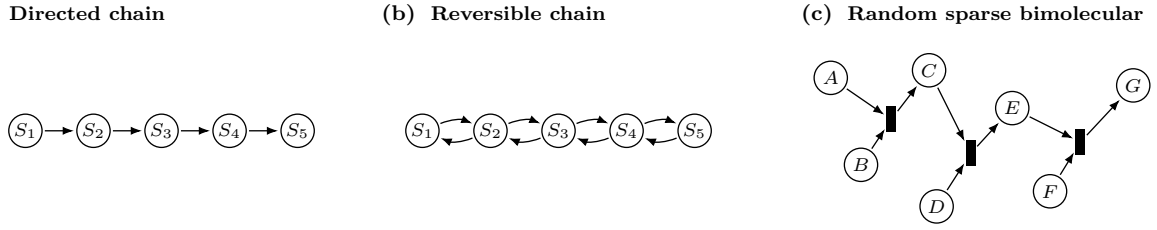


Fig. S2: Network families used in the scaling benchmark. **(a)** Directed chain with one-way conversion steps. **(b)** Reversible chain with paired forward and reverse steps. **(c)** Random sparse bimolecular network with two substrates per reaction and sparse connectivity. Circles denote species and bars denote reactions.

S4 BioModels: structural SBML interoperability

The eleven entries in Table S2 benchmark structural `SBML` exchange across a size-stratified selection from `BioModels`: ten models contain reaction structures and pass export and re-import checks without loss of stoichiometric multiplicities, while `BIOMD0000000562` serves as a reaction-free control. Across the selected models, `SynKit` preserves topological reaction networks and stoichiometric multiplicities. As an independent check, `analysis/scripts/crnt_from_xml.py` recomputes complexes, linkage classes, rank and deficiency

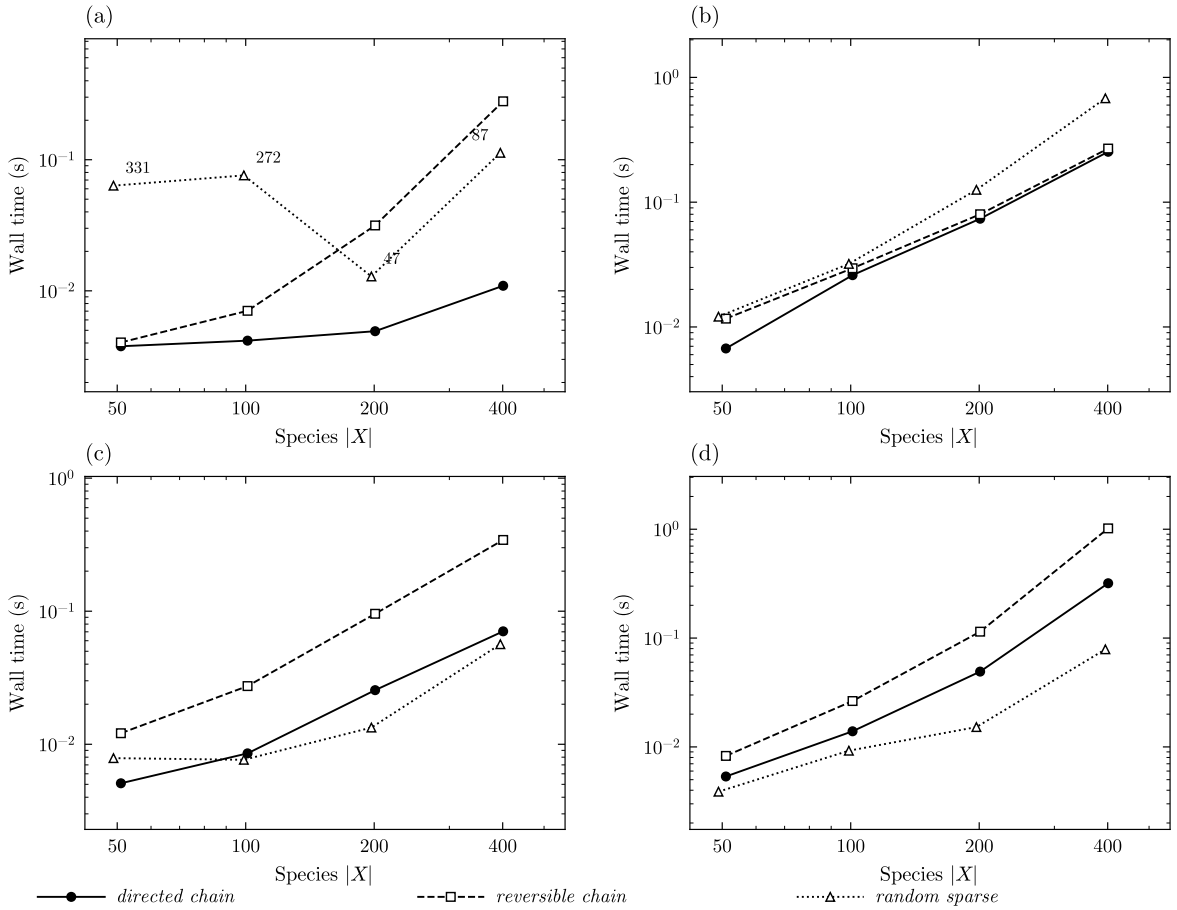


Fig. S3: Structural analysis runtimes across directed-chain, reversible-chain, and random-sparse networks. **(a)** Minimal-siphon enumeration, where numbers beside random-sparse points give minimal siphon counts. **(b)** Exact CRNT analysis. **(c)** Graph canonicalization. **(d)** Exact conserved-pool enumeration. Points and connecting lines show completed single-run measurements. Line styles and markers distinguish the three families. All 60 measurements complete.

directly from each `SBML` reaction list, using only the Python standard library and exact rational arithmetic; it agrees with Table S2 on all ten reaction-bearing models.

Table S2: BioModels input inventory and structural results. m: species, r: directed reactions, s: exact rank, δ: deficiency, P: minimal conserved pools, S: minimal siphons, RT: supported structural round trip, and Covered: every minimal siphon contains a conserved-pool support. Coverage is the sufficient persistence condition used in the main text, including the vacuous case of no siphons. Failure does not prove extinction. Dashes mark analyses inapplicable to the reaction-free control. Row `BIOMD0000000637` corrected (10 Nov): s, δ and P are 9, 9 and 3, previously 6, 0 and 6, after a fix to the `SBML` import of reversible reactions. **[To do S2.]** Regenerate this table from the `SynKit` release (main-text note 10), and state the reading of an omitted `reversible` attribute here (report C1).

Model	m	r	s	δ	P	S	RT	Covered
`BIOMD0000000001`	12	17	11	0	1	1	yes	no
`BIOMD0000000010`	8	10	5	0	3	3	yes	yes
`BIOMD0000000051`	18	48	18	8	0	0	yes	yes
`BIOMD0000000064`	26	17	17	0	11	8	yes	no
`BIOMD0000000108`	9	17	7	8	2	2	yes	yes
`BIOMD0000000404`	29	35	19	13	10	10	yes	no
`BIOMD0000000637`	12	35	9	9	3	3	yes	yes
`BIOMD0000000019`	100	125	81	41	19	20	yes	no
`BIOMD0000000175`	120	102	94	6	30	26	yes	no
`BIOMD0000000255`	504	827	486	317	17	75	yes	no
`BIOMD0000000562`	0	0	–	–	–	–	–	–

Table S3: Catalyst conventions in the BioModels entries. Mod.: number of SBML modifier references, which the reader does not import. δ is the deficiency with species listed on both sides kept (as imported), with those species cancelled as in net-stoichiometry tools, and with every modifier added to both sides of its reaction. The reaction-free control BIOMD0000000562 is omitted. Input files are those of Table S2, checked by SHA-256.

Model	Mod.	δ kept	δ cancelled	δ with modifiers
BIOMD0000000001	0	0	0	0
BIOMD0000000010	5	0	0	5
BIOMD0000000051	5	8	8	13
BIOMD0000000064	10	0	0	0
BIOMD0000000108	2	8	8	8
BIOMD0000000404	2	13	11	15
BIOMD0000000637	0	9	9	9
BIOMD0000000019	0	41	41	41
BIOMD0000000175	142	6	6	8
BIOMD0000000255	0	317	317	317

S5 KEGG: module structure and glycolysis controls

Table S4 summarizes structural deficiency and conserved moiety pools across four KEGG pathway modules: glycolysis (M00001), pyruvate oxidation (M00307), citrate cycle (M00009), and pentose phosphate (M00004). Conserved pools isolate nucleotide pairs, quinones, ferredoxins, and coenzyme A without manual curation.

Table S4: Structural results for four reduced KEGG modules. m: species, r: directed reactions, n: complexes, ℓ: linkage classes, s: rank, $\delta = n - \ell - s$: deficiency, P: minimal conserved pools, and S: minimal siphons.

Module	m	r	n	ℓ	s	δ	P	S
M00001	14	15	16	4	11	1	3	3
M00307	13	8	14	7	6	1	8	6
M00009	22	12	23	11	12	0	12	9
M00004	12	12	14	4	10	0	2	4

S5.1 Glucose-to-pyruvate pathway realization and invariants

Table S5 lists the reaction multiset converting one glucose to two pyruvates across a preparatory phase (reactions firing once) and payoff phase (firing twice). Aldolase (R01068) and triosephosphate isomerase (R01015) produce two glyceraldehyde 3-phosphate equivalents, directing payoff enzymes (R01061–R00200) to fire twice. State-space search yields a certified 15-step sequence (Fig. 5b **[To do S3.]** use Fig. 5b so the number follows renumbering.) preserving the weighted three-carbon backbone ($2 \times$ hexoses $+ 1 \times$ trioses $= 2$).

Table S5: Non-zero firing counts of the prescribed KEGG glycolysis flux. Identifiers match M00001 reaction records.

Reaction	Count	Reaction	Count
R01786	1	R01061	2
R13199	1	R01512	2
R00756	1	R01518	2
R01068	1	R00658	2
R01015	1	R00200	2

S5.2 Generic compound identifiers and cofactor boundary controls

KEGG writes polyphosphate glucokinase (R02189, EC 2.7.1.63) as C00404 + C00267 <=> C00404 + C00668. The identifier C00404 names a polyphosphate chain of any length, so $(\text{Phosphate})_n$ on the left and

(Phosphate)$_{n-1}$ on the right share it and the polymer appears on both sides. Counting phosphorus shows that one phosphate moves from the polymer to glucose: polyphosphate is a phosphoryl donor, not a retained catalyst. `SynKit` keeps the equation as written, so in the reduced glycolysis model the polymer is a species that this reaction never consumes. We therefore do not use `R02189` as an example of participant retention; the main text uses the catalysts of `BIOMD0000000404` instead (Table S3). [removed: Table on shared-polyphosphate control (participant retained versus cancelled)]

Table S6 tests execution across currency metabolite boundaries. Removing currency metabolites confirms topological realizability of the carbon backbone (14 species), whereas restoring currency metabolites (24 species) arrests hexokinase at step 0 due to unsupplied ATP.

Table S6: Currency-boundary control for the prescribed full glycolytic flux from one glucose and zero tokens of all other species. The negative decision is exhaustive for this reaction multiset.

Currency metabolites	Species	Reactions	Decision	Certificate length
Removed	14	15	realizable	15
Retained	24	15	unrealizable	–

S6 Formose networks: expansion and precursor requirements

The formose reaction system serves as a benchmark for rule-based graph transformation, exact mass conservation, and discrete pathway realizability in open-ended reaction spaces. Graph transformation explores carbohydrate condensation from formaldehyde (C_1) and glycolaldehyde (C_2) seeds through recursive aldol coupling and enediol-mediated keto-enol tautomerization.

S6.1 Network growth and stoichiometric carbon conservation

Table S7 reports network growth from the formaldehyde and glycolaldehyde seeds. After four iterations, the network contains 37 species and 44 reactions and reaches species with ten carbon atoms (C_{10}). Exact nullspace analysis of its stoichiometric matrix identifies a single positive conservation semiflow whose weights match the carbon atom count of every species, confirming carbon conservation across the generated reactions.

S6.2 Comparative rule expansion benchmarking

Table S7 compares the full API runtime of `SynReactor` DPO against `MØD` and `MolFoundry` across six expansion depths from fresh formaldehyde and glycolaldehyde seeds. Timings are evaluated across 20 timed repetitions after three warm-ups on an isolated logical CPU. Each tool uses its native rule format, and generated molecules are canonicalized without explicit hydrogens to evaluate exact network equivalence. Across all completed depths, all three tools generate identical species sets and directed stoichiometric reactions, validating algorithmic equivalence. At depth seven, all three tools exceeded system resource limits (memory or search bounds).

S6.3 Route definitions and finite precursor inventories

The targets are an aldotetrose (T_4, `O=CC(O)C(O)CO`) and an aldohexose (T_6, `O=CC(O)C(O)C(O)C(O)CO`). Under qualitative set reachability, both are reachable from the two seed species. Table S8 distinguishes qualitative depth from the reaction multiplicities required for discrete firing, and Fig. S4 shows the reaction motifs.

Table S7: Full API runtime comparison. **[To do S4.]** The `SynReactor` C++ DPO column was produced with code that is not in the `SynKit` 1.6.5 release; rerun from the new release (main-text notes 3, 8, 10; report A4). Times are median cumulative seconds from seeds over 20 timed runs after three warm-ups. Native `SynReactor` uses its C++ controller and DPO kernel with implicit hydrogen counts, whereas `MØD` and `MolFoundry` use explicit hydrogen vertices. Rule preparation and network construction are included for every tool. Counts describe identical completed species and stoichiometric reaction sets. Each expansion has a 180 s time limit and an 8 GiB per-process address-space limit.

Depth	Species	Reactions	`SynReactor` C++ DPO	`MØD`	`MolFoundry`
1	3	1	0.0152	0.000634	0.00233
2	5	4	0.0154	0.000778	0.00397
3	9	10	0.0157	0.00118	0.00523
4	37	44	0.0177	0.00291	0.0158
5	302	371	0.0417	0.0251	0.16
6	10572	11239	1.36	1.57	11.85

Table S8: Qualitative depths and prescribed discrete reaction multisets for the selected formose targets. Set reachability reuses a species once generated, whereas discrete firing requires a separate token for each consumed molecule.

Target	Qualitative depth	Discrete firings	Prescribed multiset
T_4	2	2	$r_1 + r_3$
T_6	3	4	$2r_1 + r_3 + r_7$

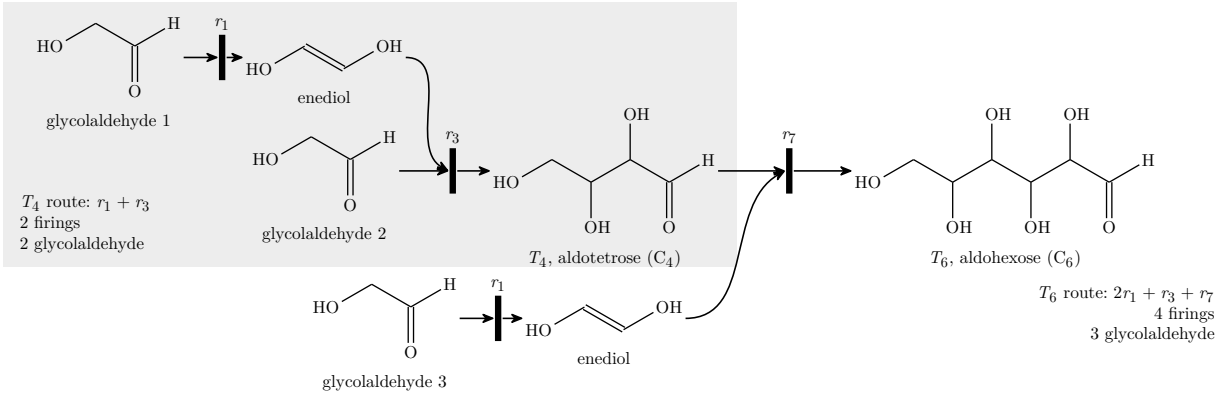


Fig. S4: Precursor requirements of the selected formose routes. Bars are reactions, as in the Petri nets of the main text, and each glycolaldehyde is drawn as a separate molecule because discrete firing consumes one token per molecule. Keto–enol tautomerization (r_1) supplies the enediol consumed by each aldol addition (r_3, r_7). The shaded block is the T_4 route ($r_1 + r_3$: two firings, two glycolaldehyde), and the whole scheme is the T_6 route ($2r_1 + r_3 + r_7$: four firings, three glycolaldehyde). Structures are constitutional, with no stereochemistry or atom mapping assigned.

Table S9 evaluates five initial precursor inventories. Formaldehyde is C_1 and glycolaldehyde is C_2. Every positive sequence is confirmed by a replayable firing sequence, while negative decisions are established by exhaustive search over the prescribed multiset. For an independent check, the same reaction multisets and initial markings are supplied to `MolFoundry`'s `FlowSolution`, without optimizing a new hyperflow. Its `is_realizable()` method and SMT certificate enumeration agree with all ten `SynKit` decisions. All three positive certificates are independently replayed, checking enabling before every firing, the requested reaction multiplicities, and target production. This comparison concerns the prescribed routes, rather than unrestricted target reachability. **[To do S5.]** Explain that the two routes consume no formaldehyde, so equal-carbon inventories differ only in glycolaldehyde count (main-text note 9; report B2).

Table S9: Realizability of the prescribed tetrose and hexose routes across five precursor inventories. Counts denote molecules and total carbon counts carbon tokens. Yes indicates a verified certificate and No an exhaustive negative decision for the selected multiset.

Pool	C_1 count	C_2 count	Total carbon	T_4	T_6
M_1	1	1	3	no	no
M_2	2	1	4	no	no
M_3	4	1	6	no	no
M_4	2	2	6	yes	no
M_5	0	3	6	yes	yes

The tetrose route converts glycolaldehyde to enediol (r_1) and couples that intermediate with a second glycolaldehyde (r_3). This motif requires two glycolaldehyde precursors and consumes no formaldehyde, so inventories M_4 and M_5 support it while M_3 (containing only one glycolaldehyde) cannot. The hexose route requires two successive aldol couplings consuming two enediol equivalents, demanding three glycolaldehyde precursors in total ($2r_1 + r_3 + r_7$), which only pool M_5 satisfies. Consequently, the three six-carbon inventories (M_3, M_4, and M_5) produce divergent realizability outcomes despite possessing identical total carbon budgets.

References

[1] Feinberg M (2019) Foundations of Chemical Reaction Network Theory, Applied Mathematical Sciences, vol 202. Springer, Cham, doi:10.1007/978-3-030-03858-8

[2] Edelstein BB (1970) Biochemical model with multiple steady states and hysteresis. Journal of Theoretical Biology 29(1):57–62, doi:10.1016/0022-5193(70)90118-9

[3] Wang L, Sontag ED (2008) On the number of steady states in a multiple futile cycle. Journal of Mathematical Biology 57(1):29–52, doi:10.1007/s00285-007-0145-z

[4] Shinar G, Feinberg M (2010) Structural sources of robustness in biochemical reaction networks. Science 327(5971):1389–1391, doi:10.1126/science.1183372